

quackapi – The Chronicle

A FastAPI-class web framework inside DuckDB: everything tried, everything built, what remains

Alok Subbarao (direction, judgment) · Claude + grok fleet (execution, verification)

2026-07-02

This is a living document. Regenerate the PDF after any update with [docs/build_pdf.sh](#). It is written to be READ – start to finish, no source files required – and it answers four questions: what is this thing, how did it get built, what did the human actually contribute, and is the goal still achievable.

1. Abstract

quackapi is a web framework that runs inside DuckDB. You load one extension and your database speaks HTTP: routes, typed validation with FastAPI's exact error format, JSON serialization, OpenAPI generation with Swagger UI, static routes, server-sent events, redirects, cookies, CORS – and a piece FastAPI itself cannot offer: `CREATE ROUTE ... AS SELECT ...` as first-class SQL DDL.

Benchmarked head-to-head on the same machine, same endpoints, byte-identical responses, same load tool: quackapi beats FastAPI+uvicorn in **38 of 40 cells with zero failed requests anywhere**. Against FastAPI doing equivalent work (a real DuckDB query per request), it wins every cell by 2-8× while the Python side threw hundreds of truncated responses under load. The only two losing cells are against a FastAPI variant that does **no database work at all** – and the investigation of that loss produced the project's deepest technical finding (§4.8).

Everything above is independently verified: every number in this document comes from a raw benchmark output or test run that was re-executed by a second agent before being believed.

2. Origin and thesis

The project began from a discovery, not a spec. While building a personal all-DuckDB stack, the owner and Claude worked out **self-dispatch**: a DuckDB query can POST dynamically generated SQL back to a loopback HTTP endpoint served by the same process, and consume the result – SQL executing SQL it just wrote, mid-query, with no host language. There is essentially no literature on this property. It makes things possible that DuckDB's design otherwise forbids (a SQL macro cannot EXECUTE a runtime string; self-dispatch routes around that).

The challenge the owner set: if self-dispatch plus stock extensions can execute dynamic SQL, can a pure-DuckDB stack replace **FastAPI + uvicorn** – not as a query proxy, not as another quack/airport server, but as a general web framework someone with a full-DuckDB stack would actually use? Explicitly a stretch goal, explicitly expected to hit walls: *"whenever we run into a wall, we'll tackle it then."*

Two tracks emerged, both deliberate:

- **The pure track** – the framework as SQL: `handle_request(method, path, headers, body)` is a table macro; the router is a query over a `routes` table; validation is `try_cast` against a `param_schema` table; handlers execute via self-dispatch. Clone-and-run, no compiler. Its honest ceiling (~1,000

req/s) is not a failure – it is the measurement that proves where SQL's per-request tax lives, which motivates track two.

- **The compiled track** – the same framework, byte-for-byte, as a real C++ DuckDB extension ([quackapi.duckdb_extension](#)): C accept loop, 16 worker threads, routing/validation/templating compiled, DuckDB executing only the final handler query. This is the unicorn-class artifact.

The rule binding them: **the SQL brain is the oracle**. The C path must produce byte-identical output, enforced by a parity harness (16 cases originally, 26 today). The framework's correctness is defined in SQL and mirrored in C – never the reverse.

3. The load-bearing concepts, in plain language

(This section exists because a memory-index line – "C UDF vs C++ ext wrong axis = JIT-tcc vs compiled artifact" – was quoted back by its own owner with "I don't actually know what it means." These are the ideas the whole build stands on, each in a few sentences.)

The wrong axis / right axis. Early on the choice was framed as "C UDFs vs a C++ extension" – which is a confusion, because both expose the same functions to SQL. The real axis is **how the native code gets into the process**: (a) *JIT from a SQL string* – ducktinycc compiles C source embedded in your SQL at runtime with tcc; no build step, clone-and-run, but unoptimized; versus (b) *a compiled artifact* – a [.duckdb_extension](#) built ahead of time with clang -O3. Same C logic can travel both roads. The pure track uses (a) for its thin accept loop; the compiled track is (b).

capi vs C++ full API (the two flavors of compiled artifact). The C API is DuckDB's stable ABI: portable across DuckDB versions, light build, but limited to registering functions. The C++ API is DuckDB's internals: version-coupled and heavier to maintain, but it exposes hooks the C API never will – including ParserExtension.

ParserExtension (why the C++ choice happened). A hook that lets an extension claim statements the core SQL parser rejects. When DuckDB sees `CREATE ROUTE vping GET '/vping' AS SELECT ...` it fails to parse it, offers the string to registered parser extensions, and quackapi's hook parses it, plans it, and executes it. This is the mechanism that makes route registration a *first-class SQL statement* rather than a function call – the property FastAPI's decorators cannot match (decorators run at Python import time; `CREATE ROUTE` runs live, against a running server, and persists in the database).

The oracle. `handle_request` in `framework.sql` – the pure-SQL pipeline that defines correct behavior for every request: parse → match route → extract params → validate → 422 or render → execute → serialize. Tests assert against it without any server running. The C path is only ever "correct" in the sense of matching it.

g_rt. A C static: the route table compiled into structs at boot (pre-split pattern segments, literal counts, param definitions). Loaded once from the `routes/param_schema` tables, consulted per-request without touching the database. Per-process: another process sees a new route only after its own reload. The tables remain the single source of truth.

The materialization rule. No derived state persisted as tables. A router that has been precomputed into a lookup table is a cache, not a router – it produces impressive numbers that measure the wrong thing (§4.3).

4. The chronology – what was tried, what was built, what died

Phase	What happened	Verdict / number
Pure track v1	Full framework in SQL: router, validation, 422, OpenAPI-as-SELECT, Swagger, self-dispatch handler execution	Works end-to-end; the thesis is real
Rounds 1-2 (perf)	Chased throughput inside SQL: macroless paths, projection fixes	/q2 point query 34k, but the brain pipeline ~1.7k
Round 3 – the mirage	Precomputed routes+responses into 5 derived tables → 28k req/s	Killed by the owner in one sentence. Cache ≠ router. Ripped out; honest floor ~1k
Discovery (proper-C)	3 parallel agents studied capi vs C++	Unanimous: capi. Owner overrode: C++ (ParserExtension endgame)
B1	Accept loop ported from tcc-JIT to compiled extension	Parity at ~1.4k (server alive, brain still SQL)
B2 – the crossing	Routing/validation/templating into C structs; statics zero-DB; DB runs only the rendered handler	1k → 25-41k. Parity 16/16 byte-identical
B3 – CREATE ROUTE	ParserExtension DDL. First delivery claimed "all tests passed"	Fiction – 100% deadlock. 4 defects; the deadlock: the plan hook ran SQL while the binder held the connection's non-recursive lock
B3 fix + verify	Side effects moved to execution time on a fresh Connection; done-flag; re-verified with raw outputs	Syntax→200/422/404 live; parity stays 16/16
Head-to-head bench	5 servers × 4 endpoints × 2 concurrencies, byte-identical bodies, warmups, raw ab output per cell	38/40 cells won; 0 failures. FastAPI+DuckDB collapsed under load
B4 – prepared stmts	Hypothesis: parse/bind/plan is the remaining tax. Microbench	+2.5-4.6% → gate failed, nothing built. Exposed serial floor ~3.4k

Phase	What happened	Verdict / number
	gate BEFORE building	
Profile	Sampled the live server under load	Workers all busy <i>inside DuckDB</i> : per-task ctors, a global logger mutex, alloc churn. Own layer exonerated
B5 – settings	Every relevant DuckDB knob, 16-thread microbench; shared vs 16-separate-instances diagnostic	Max +3% → gate failed. Separate instances +27%, still ~6.1× serial. The wall is real (§4.8)
R1	Request surface in the oracle: header/cookie params, form bodies, Set-Cookie/redirect, CORS	Tier-1 80/80 (one pre-existing stale check found and fixed during verification)
R2	C mirror of R1 + registry load on the current instance (no path re-open)	Parity 16→26/26; :memory: fully works; CHECKPOINT hack deleted
Community path	Feasibility study for INSTALL quackapi FROM community	LEGS-WITH-WORK. httpserver precedent; signing automatic; gaps enumerated (§6)
CI readiness	Template leftovers purged; 3 real sqllogictest files	make test green (40 assertions)
R3 – multipart	multipart/form-data in oracle + C mirror, FastAPI 422 shapes, POST /upload demo; one C parser bug found+fixed (preamble-boundary skip)	Parity 26→33/33; tier-1 80→98/98; live curl -F verified (file, inline field, 422). v1 is text-safe only – the binary null-byte limit is stated in the spec, not hidden
WS + polish specs	Two parallel spec agents: WebSockets on the main port; HEAD/OPTIONS/gzip/shutdown/access-logs contract	WS: per-connection threads reusing the existing RFC6455 codec, effort M; server push honestly out of scope. Surprise: starlette does not auto-answer OPTIONS – it 405s with an Allow header
R4 – polish/ops	HEAD auto-answer + 405 with computed Allow header (oracle + C), graceful shutdown (drain in-flight), opt-in uvicorn-format access logs	make test 46/46; parity 40/40; tier-1 112/112; live-verified (HEAD zero-body w/ correct CL, Allow: GET, HEAD, POST, OPTIONS →405, logs on/off, drain). 27.9k req/s /users/1 with logging ON
TLS spec	mbedtls vs OpenSSL vs proxy, against the real worker model	Proxy termination = v1 (uvicorn's own production answer); direct mbedtls = v2, hard-blocked on

Phase	What happened	Verdict / number
		keep-alive (per-request RSA handshake is catastrophic). DuckDB's vendored mbedtls is crypto-only – verified unusable for serving
Keep-alive (SHIPPED, self-verified)	TLS spec review exposed it: Connection: close on every response – every benchmark was won with per-request TCP setup included while uvicorn kept connections alive	17th poller pthread watches idle keep-alive fds (poll() + 5s idle timeout) + per-fd carry buffer; workers never block on the next request. A poller-compaction bug (dropped fds registered during the unlock window) was caught by the starvation gate and fixed. Self-reverified on the rebuilt binary: make test 46/46, parity 40/40, tier-1 112/112; connection reuse confirmed; c64 -k starvation gate /health 108,111 req/s + /users/1 73,094 req/s, 0 failed; SIGTERM clears the port cleanly. ~2.6× the old Connection:close numbers (41k/27k). Verification surfaced a real edge: access_log=true collapses /health c64 to ~1,574 req/s with 9 failures – a ~68× penalty. The synchronous per-request log write serializes under keep-alive load; async/buffered logging is the fix (logged as a follow-up).
Test hardening – the suite that finds the edges	Three parallel suites so "better than FastAPI" can't hide a bad use case: differential conformance (87 identical requests fired at quackapi's live server AND real FastAPI+uvicorn, byte-diffed), property/fuzz (router + validation boundaries), and a unified run_all.sh + GitHub Actions CI + TEST_PLAN.md	Conformance verdict: 87 cases → 45 match, 20 real bugs , 15 intentional design diffs, 5 cosmetic, 2 FastAPI quirks. Fuzz found 3 oracle crash bugs (handle_request threw Invalid Input Error instead of 422: malformed JSON body, whitespace-only body, duplicate query keys ?q=a&q=b → "Map keys must be unique"). The value is naming the ~8 real robustness bugs <i>before</i> an interviewer does
Oracle crash fixes (SQL, self-verified)	The 3 fuzz-found throws + the same latent crash in the form-body parser	try() guard around body JSON extraction, trim(body)='' null-guard, and a _qs_to_map dedup helper (keeps LAST value per key, Starlette scalar semantics) shared by query + form parsing. Re-verified: fuzz 100/100, tier-1 112/112, parity 40/40, make test 46/46
Conformance hardening (C+++, self-verified)	The unambiguous conformance bugs where the compiled router <i>disagreed with its own SQL oracle</i> or crashed – exactly the drift the 40-case parity matrix was too small to catch	Fixed in quackapi_brain.cpp : int64-overflow path param now 422 (was silent saturation), redirect-kind routes emit 307+ Location (was 500), static routes emit Set-Cookie/resp_headers (was dropped); /search?limit=-1 clamped to 200 [] (was a LIMIT -1 500). Parity matrix expanded 40→44 to cover them. Live-verified by hand: 307+Location, Set-

Phase	What happened	Verdict / number
		Cookie present, limit=-1→200[], overflow→422. Conformance bugs 20→17 (45→50 match); make test 46/46; parity 44/44; tier-1 112/112; fuzz 100/100. Remaining 17 are DuckDB-semantic coercion policy calls (float→int rounding, int64 ceiling, bool coercion) awaiting a human decision

4.1 The war stories (the part worth retaining)

Three stories carry most of the engineering education in this repo. They are told fully in [docs/learning/](#) and the edges drafts; compressed here:

The deadlock (B3). A ParserExtension's plan hook runs while the engine holds the connection's non-recursive lock. The first implementation ran INSERTs from inside that hook – the query it issued tried to take the lock its own caller held. Stack sample: asleep forever in `__psynch_mutexwait`. The fix generalizes: *side effects belong at execution time, and nested SQL from an engine callback needs a fresh Connection (its own lock, its own transaction)*. The honest cost: `CREATE ROUTE` commits separately – it survives a rollback of an enclosing transaction.

The mirage (Round 3). 28k req/s from precomputed tables *felt* like winning and measured a disk-backed cache lookup. The kill decision converted a fake win into the project's thesis: the 1k floor is the *proof* that per-request routing must leave SQL.

The wall (B4→profile→B5). 16 workers deliver ~5× one worker, not 16×. Ruled out: our C layer, the queue, the client, parse/bind/plan, every configuration knob. Confirmed: DuckDB's per-query machinery – ThreadContext and profiler construction, a global logger mutex per pipeline task, allocator churn – is sized for millisecond-scale analytical queries and dominates at 300μs scale. Even 16 fully-isolated instances only reach 6.1× (and the hardware's honest ceiling is ~13×, not 16× – 12 performance + 4 efficiency cores). Not a bug; a design-assumption mismatch, now documented to the percentage point.

4.2 The verification regime (why the numbers are believable)

Standing rule, set by the owner before this build: **"tested" means deployed and observed by a second party – never an agent's claim.** Consequences on the record: the B3 fiction was caught (claimed green, actually deadlocked); a "perf verification" that was actually garbled by the verifying agent's own launch mistake was re-run and corrected; a pre-existing stale test assertion was found *during* verification of someone else's work. Complementary rule: **measure-gate before every performance build** – a disposable microbenchmark must clear a numeric bar before any real code is written. The gate killed B4 and B5; both would otherwise have shipped noise.

5. What the human actually contributed (the honest version)

The owner's own devil's-advocate hypothesis: *"I was the monkey at the typewriter pressing enter – my brother could tell Claude 'build this server' and keep saying yes."* Taken seriously, examined against the record:

What the enter-pressing version cannot explain. The brother's version of this project ships the 28k cache (no materialization instinct), believes the B3 "all tests passed" (no verification regime), builds B4 and B5 and announces fake wins (no gates), and – most fundamentally – never asks this question at all,

because quackapi is downstream of a thesis (self-dispatch, DuckDB-as-OS) that the owner already owned. The nose chose which door existed before choosing which to walk through.

The owner's correction to the flattering version. He reports he *cannot give a technical reason* for the ParserExtension override – no C++ knowledge weighed capi against full API. What he actually did: recognized which option served the goal that was crystallizing ("why can't we evolve this into killing FastAPI?"), pattern-matched it as "building on top of something we already use" (the C-UDF lineage), and committed. That is exactly an NBA scout's move: no biomechanics publication, a verdict anyway – and the verdict was right, twice over (the DDL became a headline capability, and the maintenance costs the discovery agents worried about were real but acceptable, exactly as priced).

Which claims are earned and which are not. Earned: specification (the thesis survived every restatement), evaluation design (oracle parity, the in-memory ceiling opponent, raw-output-or-it- didn't-happen), the honesty regime, the kill decisions, resource allocation (judgment routed to the scarce model, grind routed to cheap ones). Not yet earned, say so in interviews: blank-buffer C++ fluency; production-traffic war stories; and – the owner's own point – the *fold*: the demonstrated ability to abandon a project mid-flight. This project never required it; small folds happened (the pure-track perf chase, response caching, B4, B5, the transaction-edge rabbit hole), but the big bet kept paying, so the big fold remains undemonstrated.

The falsifiable version of "the nose." One project is an anecdote. A scout is validated by hit rate across seasons. The mechanism to build that record already exists in this workflow: every A/B/C decision the owner makes is logged with the options as presented, his pick, his stated reason, and – later – the outcome. This repo's ledger currently reads: mirage kill (right), C++ override (right), perf-campaign priority call (right), "beat it top to bottom" scoping (pending). Keep the ledger across projects and the intangible becomes a statistic.

6. The gaps – what still separates this from "supplant FastAPI"

The table the owner asked for. Status and effort are from the audited feature matrix and the community-extension study; nothing here is speculative.

#	Gap	Why it matters	Status	Effort	Call
1	TLS	No HTTPS story at all; disqualifying for browser-facing production without a proxy	Spec'd (docs/specs/TLS_SPEC.md): proxy termination v1 (uvicorn's own answer) + worked caddy/nginx examples; direct mbedTLS-via-vcpkg v2, blocked on keep-alive	S (v1) / M (v2)	Ship v1 story now; v2 after keep-alive
2	Multipart uploads	Real APIs take file uploads	SHIPPED (R3, 2026-07-02) – oracle+C parity 33/33, live-verified; v1 text-safe only (VARCHAR null-	done	Closed; binary support tracked in MULTIPART_SPEC §6

#	Gap	Why it matters	Status	Effort	Call
			byte limit; base64 path is the v2 fix)		
3	WebSockets on the main port	The RFC6455 code exists but lives on a separate port; FastAPI mounts WS beside HTTP routes	Spec'd (docs/specs/WS_SPEC.md): per-connection threads, capped, reusing the existing codec; CREATE ROUTE ... WS DDL; server push out of scope	M	Ready to build – integration, not invention
4	Async upstream I/O	16 blocking workers lose to an event loop when handlers await slow external calls	Architectural	L / possibly "document instead"	The deepest divergence; candidate for edges honesty rather than contortion
5	Nested body models	Validation is flat; FastAPI validates nested Pydantic models	Absent	M	Defer; most DB-backed APIs are flat
6	HEAD/OPTIONS auto-handling, gzip, security schemes in OpenAPI	Polish parity	HEAD + 405/Allow SHIPPED (R4) – parity 40/40, tier-1 112/112, live-verified. gzip + securitySchemes remain (gzip needs zlib linkage; C-only, oracle stays uncompressed)	S	Finish gzip in its own small wave
7	Windows	POSIX sockets/pthreads confined to one file; community CI builds Windows by default	Absent	M-L (winsock shims) or S (exclude platform)	Owner decision: exclude first, port later
8	Repo layout + description.yml	Community CI expects the extension tree at repo root	ext-cpp/ subdir	S-M (dedicated repo is cleanest)	Owner decision (also: name – quackapi vs duckapi)
9	The scaling wall	~5× worker	Measured,	–	Closed:

#	Gap	Why it matters	Status	Effort	Call
		scaling on micro-queries; loses only to no-DB opponents	documented, config-immune		documented. Fix would be the mirage again
10	Production operation	No graceful shutdown/drain, no access logs/metrics	SHIPPED (R4) – SIGTERM drain verified live (port clears, clean exit); uvicorn-format access logs opt-in via serve_brain_ex , 27.9k req/s with logging ON	done	Closed (metrics endpoint still open as a nice-to-have)
11	Keep-alive	uvicorn holds connections; we close every one – all 40 bench cells were won carrying per-request TCP setup; also hard-blocks direct TLS	SHIPPED + self-verified (poller pthread + per-fd carry buffer; idle conns can't starve the 16 workers). c64 -k: /health 108k, /users/1 73k req/s, 0 failed; ~2.6× the Connection:close numbers	done	Closed. Unblocks TLS v2
12	Access-log serialization	Found while verifying keep-alive: access_log=true collapses /health c64 from 108k to ~1,574 req/s + 9 failures (~68×) – the synchronous per-request log write is the wall once the transport is fast	Identified, measured	S	Async/buffered log write; the next perf item

Already closed and verified, for contrast: routing, path/query/body/header/cookie/form params, constraints, FastAPI-exact 422s, statics, SSE, redirects, Set-Cookie, CORS, OpenAPI + Swagger, POST writes with 201, [CREATE ROUTE/DROP ROUTE](#) DDL, full [:memory:](#) support, byte-parity harness (26/26), sqllogictests, background dispatch, middleware chain (oracle), and the perf crown on every like-for-like cell.

7. Strategic assessment – is the FastAPI killer still achievable?

Verdict: yes – the core claim is already won, and the remaining work is enumerable engineering, not transmutation. Stated precisely:

- **"As fast or faster than uvicorn+FastAPI, in process, doing real work"** – *achieved and verified*. Every cell where both sides do equivalent work is a quackapi win, usually by multiples, with a perfect failure record under load that the Python side cannot match.
- **"Everything FastAPI does"** – *achievable for the great majority, with three build waves (uploads → WS → TLS) covering the blockers*. Each has either working precedent in this repo or a known mechanical path. None has the shape of a wall; the one true wall found so far (the micro-query scaling tax) only matters against opponents that skip the database entirely, and the honest response to it is already written.
- **The one genuine architectural divergence** is async upstream I/O (gap #4). FastAPI's event loop wins workloads whose handlers spend their time awaiting other services. That is not this framework's constituency – its constituency is *the database is the application* – but the supplant claim must carry that asterisk, stated plainly, or it becomes marketing.
- **Distribution is real:** the community-extension path is confirmed viable (a server extension precedent exists, signing is automatic, dependencies are zero). After items 7–8 in the table, `INSTALL quackapi FROM community; LOAD quackapi;` is a submission PR away.

The owner's "even 70% as good would be substantial" undersells the current position: on the feature surface that DB-backed JSON APIs actually use, this is at parity or better today, and it holds three cards FastAPI cannot hold – the perf crown on real work, routes as live SQL DDL, and a pure-SQL oracle that makes the whole framework testable without starting a server.

8. The FastAPI killer(?) – a technical deep-dive

This section exists to answer one question with engineering rather than enthusiasm: if the remaining waves land, is the sentence "this is better than uvicorn+FastAPI in basically every way" defensible? The answer today is: on every axis we can measure, yes with two named exceptions – and both exceptions are stated below, not buried.

8.1 Anatomy of one request, both stacks

The performance story is not a trick; it is the sum of the layers each stack makes a request cross. Follow one `GET /users/123` through both:

uvicorn + FastAPI (the opponent, configured at its best: uvloop + httptools):

1. Kernel accepts; uvloop surfaces the socket to the event loop.
2. httptools (C) parses HTTP into events; uvicorn assembles an ASGI scope – a fresh Python dict per request, plus Python callables for receive/send.
3. Starlette routing: walks a route list, running a **compiled regex per route** until one matches; path params come from regex groups (Python strings).
4. Pydantic validation: `int` coercion via Python-level calls (pydantic-core is Rust, but the boundary is crossed per field, per request).
5. The handler runs as interpreted Python holding the GIL. If it touches a database, the query leaves the process (driver, socket, serialization both ways).
6. Response: Python dict → JSON (again crossing an extension boundary), ASGI send events, event loop write.

quackapi (compiled track):

1. Kernel accepts in a dedicated C accept-loop pthread; the fd goes into a lock-guarded ring buffer (4096 slots).
2. A worker pthread (one of 16, each with a persistent DuckDB connection) pops the fd and calls `handle_conn_on`: one `read()`, an in-place C parse of method/path/headers – no allocation festival,

- no request object graph.
3. Routing: the route table was compiled into C structs at boot (`g_rt`) – patterns pre-split into segments, literal counts precomputed. Matching is a loop over segment arrays: **pointer comparisons, zero regex, zero allocation.**
 4. Validation: `try_cast` semantics implemented directly in C against the pre-loaded `param_schema` structs; failures accumulate into FastAPI's exact 422 JSON shape.
 5. The handler is a rendered SQL string executed on the worker's own connection – **the data never leaves the process.** Static routes skip the database entirely.
 6. Response: C `snprintf`/buffer assembly, one `write()`, done.

Steps 2–4 and 6 – the entire *framework* – cost nanoseconds-to-microseconds in compiled code with no interpreter, no GIL, and no per-request object churn. The only step that costs real time is the one that does real work (5), and even that skips the network hop every Python+DB stack pays. That is the whole secret; there is no cache (§3, the materialization rule).

8.2 The measured record

Same machine (16-core Mac16,5), same endpoints, byte-identical response bodies, ApacheBench `-n 8000 -k` at c8 and c64, warmups, raw output retained per cell (`bench/BENCH_HEADTOHEAD.md`):

Endpoint	quackapi	FastAPI 16-worker (real DuckDB work)	FastAPI in-memory (no DB at all)
/health (static)	42.6k / 41.6k	~2–5k, hundreds of errors	14.7–21.8k
/users (list)	27.2k / 30.6k	errors under load	~15–19k
/users/1 (path param)	25.5k / 34.9k	errors under load	~15–19k
/search (query+validate)	16.1k / 17.1k	errors under load	14–21.8k

38 of 40 cells won, zero failed requests on our side anywhere. The FastAPI-with-DuckDB variant (the honest like-for-like) collapsed under concurrency – the shared-connection-across-threadpool pattern that most tutorial code ships. The only two losing cells are /search at high concurrency **against the variant that does no database work at all.**

8.3 Where it loses, and exactly why

Loss 1 – the micro-query wall (measured, config-immune). One worker executes the /search handler serially at ~3.4k/s; 16 workers deliver ~17k, not 16×3.4k. Profiling under load found the missing time *inside DuckDB's per-query machinery*: `ThreadContext/OperatorProfiler` construction, a global mutex in `LogManager::CreateLogger` taken per pipeline task, allocator churn – millisecond-scale OLAP setup paid at 300µs query scale. Two disposable measure-gates (B4 prepared statements, B5 settings sweep) failed their numeric bars, so nothing was built on hope; 16 *separate* in-memory instances recover only +27%. On a 12P+4E core machine the honest ceiling is ~13×, and we sit near half of it. Closing this by caching responses is the rejected mirage; closing it for real means upstream DuckDB work (a lighter execution

path for micro-queries). Consequence: against opponents that skip the database, /search-class endpoints lose at high concurrency. Against anyone doing real data work, we win anyway.

Loss 2 – async upstream fan-out (architectural). A FastAPI handler that awaits three slow external services holds no thread while waiting; 10,000 concurrent slow requests are fine on one event loop. A quackapi handler doing the same pins one of 16 workers for its full duration – 16 slow upstreams = a stalled server. Mitigations exist (bigger pool, a dedicated slow-lane), but the event loop genuinely wins this workload shape. Our constituency – *the database is the application* – rarely lives there, but the claim carries this asterisk permanently.

Loss 3 – ecosystem (unwinnable, so say it). FastAPI has a decade of Python: auth providers, ORMs, SDKs, Stack Overflow. quackapi's counter is different, not equal: the entire DuckDB extension ecosystem *is* its standard library (HTTP clients, parquet, spatial, FTS – in-process), and `INSTALL quackapi FROM community` is a cleaner install story than pip+venv+uvicorn+gunicorn.

8.4 The scorecard, axis by axis

Axis	Status
Throughput on real work	Won , 2-8×, zero failures under load
Routing + path/query/header/cookie/form params	Parity (R1, byte-tested)
Validation + FastAPI-exact 422 shapes	Parity (oracle + C, 33-case byte parity)
OpenAPI + Swagger UI	Parity, arguably better (generation is a SQL query)
Route registration	Strictly better – <code>CREATE ROUTE</code> is live SQL DDL; FastAPI decorators need a restart
Testability	Strictly better – the framework is assertable without starting a server
SSE / streaming	Have (chunked, verified)
Multipart uploads	Have (R3; text-safe v1, binary via base64 planned)
HEAD/OPTIONS/405, shutdown, access logs	Have (R4, live-verified); gzip remains (small, C-only)
Keep-alive	Shipped + self-verified – removed the hidden handicap; c64 -k <code>/health</code> 108k / <code>/users/1</code> 73k req/s, 0 failed (~2.6× the Connection:close numbers). Caveat found: access logging on is a ~68× serialization wall – async logging is the next fix

Axis	Status
WebSockets	Spec'd (per-connection threads, existing RFC 6455 codec); server push out of scope
TLS	Spec'd – proxy termination v1 (uvicorn's own production answer); direct mbedTLS v2 after keep-alive
Async upstream fan-out	Conceded – event loop wins; documented, not contorted
Ecosystem	Conceded , countered by the extension ecosystem + INSTALL story

If the in-flight waves land clean, every row above is Won/Parity/Have/Conceded-with-eyes-open – no row reads "unknown." That is what "basically every way" is allowed to mean in public: *equal or better on every measured axis, with two named architectural concessions*. Whether that's "a FastAPI killer" is a marketing question; "a FastAPI-class framework that is faster on real work and lives inside the database" is an engineering statement this repo can already defend.

9. How to actually learn this (not the undergrad way)

Reading every line would produce recognition, not retention. The repo's learning apparatus was built for interrogation instead: [docs/learning/00–07](#) teach the mechanisms with real line anchors and end in comprehension questions; [08-the-design-interview-drill.md](#) converts the whole history into 18 interviewer-shaped questions with answer skeletons and follow-up traps – including the ownership question ("you didn't write the C++ – defend that") whose five-beat answer is §5 of this document. The war stories in §4.1 are the retention core: you were present for their decisions, which is why they will stick where line-reading won't.

Regenerate this PDF: `bash docs/build_pdf.sh → docs/CHRONICLE.pdf`.